

ScholarshipFinder — Technical Presentation Script

Web Engineering Class

1. Introduction — What & Why

"ScholarshipFinder is a Django web application that centralizes scholarship discovery and request management for Turkish students in Japan under JTOB. The core problem: scholarship data was scattered across PDFs and emails, with no searchable interface and no workflow for students to signal interest or for admins to track requests."

2. Architecture — Django MTV Pattern

"We followed Django's Model-Template-View architecture with one Django app (`scholarships`) containing three models."

Models (`scholarships/models.py`):

- `User` — extends `AbstractUser`, uses email as the `USERNAME_FIELD` instead of username. Fields for education level, discipline, and prefecture.
- `Scholarship` — 18 fields including section, foundation, qualifier codes, award contents, duration. Composite unique constraint on (`foundation_name`, `scholarship_name`).
- `ScholarshipRequest` — joins User to Scholarship with a status workflow: pending → approved/rejected. Tracks reviewer, review date, and admin notes.

"Views are function-based, organized as: public views (home, list, detail), auth views (register, login, logout), authenticated views (profile, request form), and admin views (dashboard, request review, CSV reload)."

"Templates are server-rendered Django templates with custom CSS — no frontend framework. This was intentional: the app is content-heavy with minimal interactivity."

3. Data Pipeline — CSV → Database

"Scholarship data lives in a CSV file — this is the source of truth. The `import_scholarships` management command handles the ETL pipeline:

- **Extract:** reads the multi-section CSV with a header at row 4
- **Transform:** normalizes section codes (III/IV/V), parses `plural_grants` from raw Y/N prefixes into structured Yes/No/Unknown, strips metadata into a `notes` field
- **Load:** uses `update_or_create` with the composite unique key — so re-importing updates existing rows instead of duplicating

The admin can trigger this from the browser at `/admin/reload/`, which calls `call_command('import_scholarships', '--overwrite')`. This was a key design decision: keep data in a familiar spreadsheet format and let the code normalize it."

4. Data Cleaning Pipeline

"A separate `clean_scholarships` command runs sanity checks across every field. It strips non-ASCII garbage characters using a regex `[^\x20-\x7E\n\r\t]`, moves extraneous text into the `notes` field, and preserves clean data in the main fields. It supports `--dry-run` to preview changes before applying. This was necessary because the CSV contained encoding artifacts from its original Japanese sources."

5. Award Amount Parsing — Regex Pipeline

"Award amounts came as free-text like `'25-41,000/Y'`, `'120,000/M'`, or `'Up to 40/M'`. We built a two-tier parsing system:

1. **Server-side filtering** (`extract_max_award_value` in `views.py:22`): extracts the maximum numeric value, converts to monthly, handling ranges (takes the

upper bound), yearly-to-monthly conversion, and full-width Unicode normalization. Used by the award amount filter.

2. **Template-side display** (`transform_award_amount` in `scholarship_extras.py:214`): formats amounts for human display as `'¥50,000'`, `'¥25,000 - ¥41,000'`, or `'Variable'`.

Both handle edge cases: ranges, yearly/monthly indicators (`Y/M/年/月`), variable amounts (`'Not fixed'`, `'TBA'`), and full-width character normalization. CSV values are stored in ¥1,000 units — we multiply by 1000 during parsing."

6. Qualifier Code System

"Scholarship eligibility uses abbreviated codes: `U` for Undergraduate, `M` for Master's, `D` for Doctoral, `HS` for High School, etc. The system manages them at three levels:

1. **Storage**: raw codes stored as-is from CSV
2. **Filter form** (`forms.py:124`): dynamically builds checkbox options by scanning distinct qualifier values in the database, extracting base codes via regex, and mapping them through `QUALIFIER_MAPPING`
3. **Display** (`expand_qualifier` filter): converts codes to readable text like `'Undergraduate, Master's'`

The filter also handles complex codes like `'U(3-4)'` — stripping parentheses for matching while keeping them for display context."

7. Search & Filter System

"The scholarship list page has six filter dimensions:

- **Section** — dropdown (Local Govts, Private Foundations, Applicants Abroad)
- **Name & Designated Schools/Fields** — `__icontains` for case-insensitive substring matching
- **Qualifier** — `CheckboxSelectMultiple`, builds an OR query with `Q` objects
- **Plural Grants** — exact match on Yes/No/Unknown
- **Award Amount** — iterates over filtered results, parses each scholarship's `contents` field, and only keeps those matching the minimum

The amount filter is the most complex: it can't use database-level filtering because amounts are embedded in free text. Instead it scans the queryset in Python, parses each entry with `extract_max_award_value`, and filters client-side. Variable amounts (not fixed, TBA) are always included regardless of the minimum threshold."

8. Authentication Flow

"We use Django's built-in auth framework with a custom User model. Key decisions:

- `USERNAME_FIELD = 'email'` — login by email, no username needed
- `RegistrationForm` extends `UserCreationForm` — sets `user.username = user.email` on save to satisfy Django's `unique` constraint on username
- `CustomLoginForm` extends `AuthenticationForm` — relabels the username field to 'Email'
- Password change uses Django's class-based `PasswordChangeView` with custom templates

Security: SSL forced in production, session and CSRF cookies marked secure, CSRF trusted origins configured per environment."

9. Request Workflow — State Machine

"The request system implements a simple state machine: - `pending` → `approved` or `rejected` - Duplicate detection: a user can't submit a second pending request for the same scholarship - Admin review records: reviewer identity (`reviewed_by`), timestamp (`reviewed_date`), and notes (`admin_notes`) - The admin dashboard shows aggregate stats (pending count, total count) and the 10 most recent requests

This is intentionally simple — no email notifications, no complex state transitions. It's a lightweight CRUD workflow that replaces email back-and-forth."

10. Template Layer — Template Tags & Filters

"We built custom template filters to handle data transformation at render time:

- `expand_qualifier` — `U` → `'Undergraduate'`
- `expand_selection_method` — `'D,I'` → `'Document, Interview'`
- `expand_inquiry_method` / `expand_application_method` — `'F/S'` → `'Fax / Standard'`
- `expand_duration` — `'M: 2y'` → `'Master's: 2 year(s)'`
- `format_multiline` — *handles full-width character cleanup and newline-to-`<br>` conversion*
- `transform_award_amount` — *formats award amounts with yen signs and commas*

These keep the templates readable and the business logic testable."

11. Deployment — Render

"Deployed on Render's free tier:

- **Web service:** *Python 3.12, Gunicorn WSGI, WhiteNoise for static files*
- **Database:** *Render PostgreSQL (free tier)*
- **Build:** `pip install -r requirements.txt && python manage.py migrate`
- **Start:** `gunicorn config.wsgi --log-file -`
- **Domain:** `scholarships.jtob.org` *(custom domain with SSL)*
- **Environment:** *all secrets injected via env vars* — `SECRET_KEY`, `DATABASE_URL`, `ADMIN_PASSWORD`

The database connector uses `dj-database-url` so it defaults to SQLite in development and PostgreSQL in production — zero config changes needed between environments."

12. Code Quality

*"We enforce: - **Ruff** for linting (E, F, I, N, W rule sets) - **Black** for formatting (88 char line length, Python 3.12 target) - **pytest** for testing - All configured via*

```
pyproject.toml with single-command execution: ruff check . && black --  
check . && pytest "
```

13. Key Engineering Takeaways

1. **CSV-as-source-of-truth** — non-technical admins maintain data in a spreadsheet; the app normalizes it on import
 2. **Python-side parsing for unstructured data** — award amounts in free text can't be filtered at the database level; we scan and parse in Python
 3. **Template filters for display logic** — kept view code clean; all data transformation for display lives in `templatetags/`
 4. **Environment-driven config** — single settings file adapts via env vars between dev (SQLite) and production (PostgreSQL)
 5. **No frontend framework** — server-rendered templates + custom CSS was sufficient for a content-centric app, reducing complexity and dependencies
-

Q&A Prep

Question	Answer
<i>Why not use a JS frontend?</i>	The app is primarily read-only browsing with simple forms. Server-rendered templates give faster initial load, simpler deployment, and less code to maintain. The interactivity needed is minimal and handled by vanilla JS.
<i>How would you scale the amount filter?</i>	Currently O(n) per query. For larger datasets, we'd extract and store the award amount as a dedicated numeric column during import, enabling database-level filtering.
<i>Why function-based views instead of class-based?</i>	The views are simple enough that CBVs would add more boilerplate than they save. FBVs make the request/response flow explicit.
<i>How do you handle CSV encoding issues?</i>	The import normalizes full-width Unicode characters. The <code>clean_scholarships</code> command strips remaining non-ASCII garbage into the notes field. We log all encoding issues.

Question	Answer
<i>What would you add next?</i>	Email notifications on status changes, deadline alerts, award amount as a dedicated database column, and an API for partner organizations.